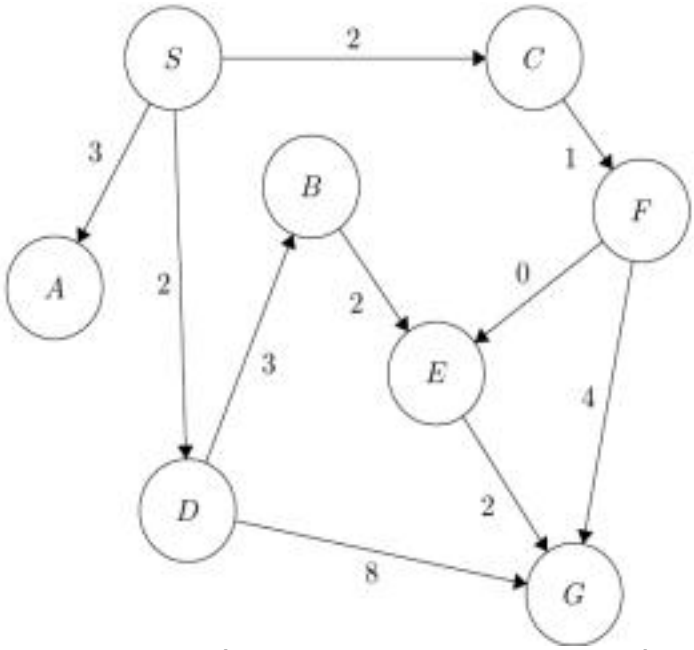
# BÀI TẬP TÌM KIẾM TRONG AI

## Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.




*Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm*

**Yêu cầu**: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

**1.1. Breadth-First Search (BFS)**

| Nội dung | Kết quả |
|---|---|
| Explored Set | $\{S, A, C, D, F, B, G\}$ |
| Path | $S \rightarrow D \rightarrow G$ |
| Total Cost | *10* |



Giải quyết vấn đề bằng tìm kiếm **2** / **8**

**1.2. Depth-First Search (DFS)**

| Nội dung | Kết quả |
|---|---|
| Explored Set | $\{S, A, C, F, E, G\}$ |
| Path | $S \rightarrow C \rightarrow F \rightarrow E \rightarrow G$ |

| | |
|---|---|
| Total Cost | *5* |

**1.3. Iterative Deepening Search (IDS)**

| Giới hạn độ sâu | Các nút được duyệt |
|---|---|
| 0 | $\{S\}$ |
| 1 | $\{S, A, C, D\}$ |
| 2 | $\{S, A, C, F, D, B, G\}$ |
| ... | |

**Path cuối cùng:** $S \rightarrow D \rightarrow G$

**Total Cost:** $2 + 8 = 10$

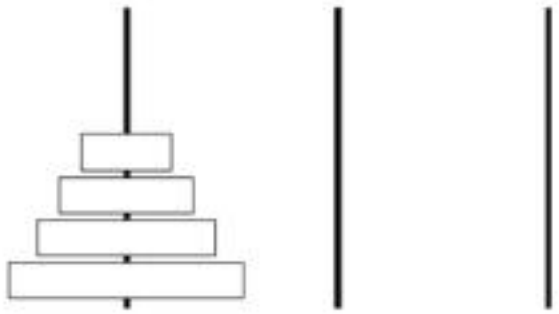
**Câu hỏi thảo luận:** BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

**Cấu trúc Frontier:** BFS sử dụng hàng chờ FIFO (Queue), trong khi DFS sử dụng ngăn xếp LIFO (Stack).

**Tính tối ưu:** BFS tối ưu về số bước/độ sâu (trên đồ thị chi phí bằng nhau), nhưng không đảm bảo tối ưu chi phí khi các cạnh có trọng số khác nhau. DFS không đảm bảo tính tối ưu ngay cả khi chi phí các cạnh bằng nhau và có thể bị lặp vô hạn trên đồ thị có chu trình.

## Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



*Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội*

**2.1. Biểu diễn trạng thái**

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

Một trạng thái được biểu diễn bằng một bộ (tuple) gồm $n$ phần tử: $s = (p_1, p_2, \ldots, p_n)$

Trong đó $p_i \in \{1,2,3\}$ biểu thị vị trí cọc (cọc 1, cọc 2, cọc 3) của đĩa thứ $i$ (với quy ước đĩa *1* là đĩa nhỏ nhất, đĩa $n$ là đĩa lớn nhất).

### 2.2. Kích thước không gian trạng thái

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

Số trạng thái có thể có: $3^n$

Giải thích: Có $n$ đĩa, mỗi đĩa có thể nằm độc lập ở *1* trong *3* cọc, do đó số lượng cấu hình trạng thái là $3 \times 3 \times \ldots \times 3 = 3^n$.

### 2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

$$S_0 = (1,1,\ldots,1)$$

### 2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?

Một hành động $a = (d, p_{from}, p_{to})$ đại diện cho việc chuyển đĩa $d$ từ cọc $p_{from}$ sang cọc $p_{to}$.

Một hành động là hợp lệ tại trạng thái $s$ khi thỏa mãn:

1. Đĩa $d$ phải là đĩa ở trên cùng (top) của cọc $p_{from}$ tại trạng thái $s$.
2. Đĩa $d$ phải nhỏ hơn đĩa trên cùng của cọc $p_{to}$ (hoặc cọc $p_{to}$ đang trống).

### 2.5. Goal Test

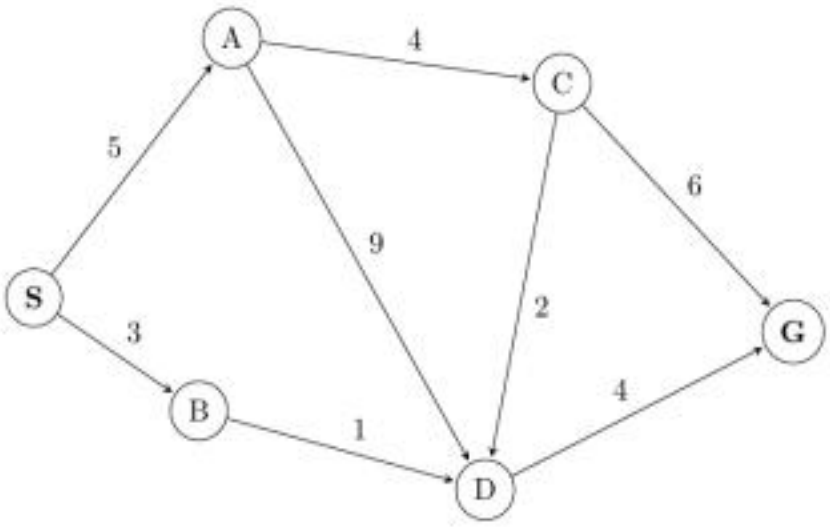
Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

Hàm goalTest(state) trả về True khi toàn bộ các đĩa đã được di chuyển sang cọc 3:

$$goalTest(s) = True \iff s = (3, 3, \ldots, 3)$$

## Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



*Hình 3. Đồ thị dùng để thiết kế heuristic*

**Công thức A*:** f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

### 3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

3

Giải quyết vấn đề bằng tìm kiếm **4** / **8**

| Node | h(n) |
|---|---|
| S | 8 |
| A | 10 |
| B | 5 |
| C | 6 |
| D | 4 |
| G | 0 |

### 3.2. Chạy A* với heuristic đã thiết kế

Mở rộng $S$: $f(S) = 0 + 8 = 8$. $S$ sinh ra $A$ ($f = 5 + 10 = 15$), $B$ ($f = 3 + 5 = 8$).
Mở rộng $B$: $f(B) = 8$. $B$ sinh ra $D$ ($f = (3 + 1) + 4 = 8$).
Mở rộng $D$: $f(D) = 8$. $D$ sinh ra $G$ ($f = (4 + 4) + 0 = 8$).
Mở rộng $G$: Đạt đích.

**Explored Set:** $\{S, B, D, G\}$

**Path:** $S \rightarrow B \rightarrow D \rightarrow G$

**Total Cost: 8**

### 3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

Đề xuất bảng $h(n)$:

- $h(S) = 0, h(A) = 0, h(B) = 0, h(C) = 6, h(D) = 0, h(G) = 0$.
- Hoặc đơn giản điều chỉnh $h(C) = 6$ và $h(D) = 0$ (vẫn $\leq h^*(n)$ nên admissible).

Cạnh vi phạm condition $h(n) \leq c(n, n') + h(n')$:

- Xét cạnh $(C, D)$ với $c(C, D) = 2$:
- $h(C) = 6, h(D) = 0$.
- Ta có: $h(C) = 6 > c(C, D) + h(D) = 2 + 0 = 2$.

⇒ Vi phạm điều kiện Consistent.

**3.4. Câu hỏi mở rộng**

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.
Giả sử $h$ là consistent, tức là với mọi cạnh $(n, n')$ ta có $h(n) \leq c(n, n') + h(n')$.

Bằng phương pháp truy hồi dọc theo đường đi ngắn nhất từ $n$ đến $G$ là $n = n_0 \rightarrow n_1 \rightarrow n_2 \rightarrow \ldots \rightarrow n_k = G$:

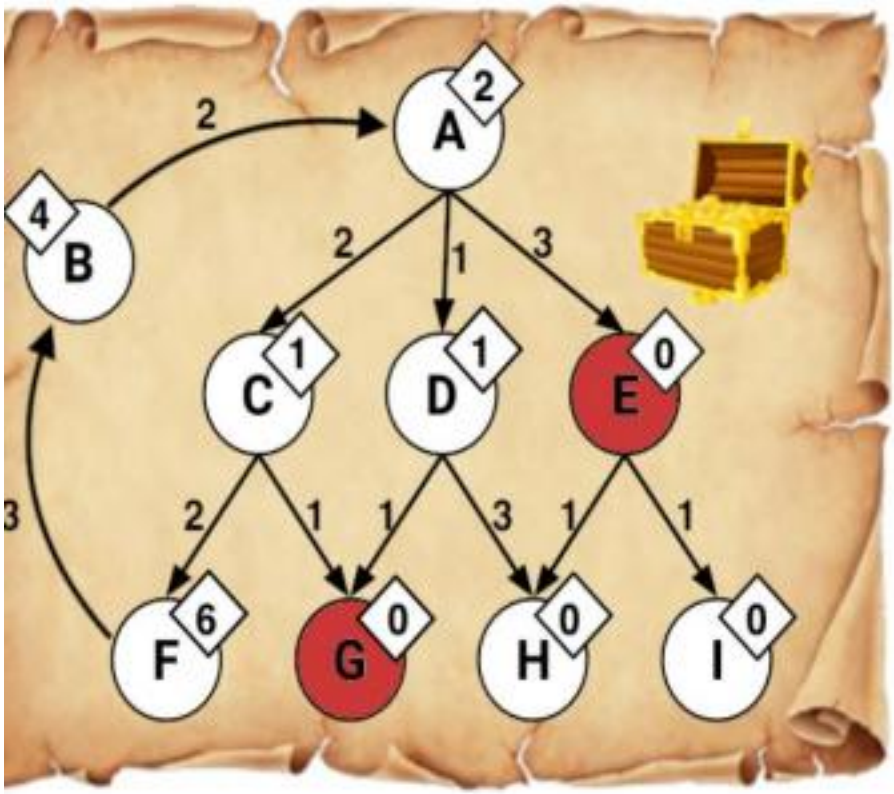
$$h(n_0) \leq c(n_0, n_1) + h(n_1) \leq c(n_0, n_1) + c(n_1, n_2) + \ldots + c(n_{k-1}, G) + h(G)$$

Vì $h(G) = 0$, tổng các chi phí $c(n_i, n_{i+1})$ chính là chi phí thực tế $h^*(n)$ của đường đi ngắn nhất từ $n$ đến $G$.

Do đó $h(n) \leq h^*(n)$, chứng tỏ $h(n)$ luôn admissible.

## Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.




*Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)*

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

**4.1. Depth-First Search**

**Explored Set:** ***{A, C, F, B, G}***
**Path returned:** $A \rightarrow C \rightarrow G$
**Cost: 3**

**4.2. Breadth-First Search**

**Explored Set:** *{A, C, D, E}*
**Path returned:** $A \rightarrow E$
**Cost: 3**



Giải quyết vấn đề bằng tìm kiếm **6** / **8**

**4.3. Uniform-Cost Search**
**Explored Set:** ***{A, D, C, G}***
**Path returned:** $A \rightarrow D \rightarrow G$
**Cost: 1 + 1 = 2**

**4.4. Greedy Search**

**Explored Set:** ***{A, E}***
**Path returned:** $A \rightarrow E$
**Cost: 3**

**4.5. A* Search**

**Explored Set: *{A, D, G}***
**Path returned: $A \rightarrow D \rightarrow G$**
**Cost: 1 + 1 = 2**

**Bảng tổng hợp so sánh**

| Thuật toán | Tiêu chí chọn node | Path | Cost |
|---|---|---|---|
| DFS | Độ sâu | $A \rightarrow C \rightarrow G$ | 3 |



Giải quyết vấn đề bằng tìm kiếm **7** / **8**

| | | | |
|---|---|---|---|
| BFS | Độ sâu nhỏ nhất | $A \rightarrow E$ | 3 |
| UCS | g(n) | $A \rightarrow D \rightarrow G$ | 2 |
| Greedy | h(n) | $A \rightarrow E$ | 3 |
| A* | g(n)+h(n) | $A \rightarrow D \rightarrow G$ | 2 |

## Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

**Câu 1.** Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

Đúng / Sai: Sai

Giải thích: Trong một số trường hợp, nếu heuristic không đủ mạnh (chẳng hạn $h(n) = 0$), $A^*$ trở thành UCS và có thể phải duyệt số lượng nút bằng hoặc nhiều hơn DFS. DFS có thể tình cờ đi đúng ngay nhánh chứa đích ở độ sâu nông và kết thúc rất sớm mà chỉ duyệt cực kỳ ít nút.

**Câu 2.** Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B. Đúng / Sai: Sai

Giải thích: Quân xe trong cờ vua có thể di chuyển nhiều ô trên cùng một hàng hoặc cột chỉ trong 1 nước đi. Do đó, khoảng cách Manhattan (đếm tổng số ô) sẽ đánh giá cao hơn số nước đi thực tế cần thiết, dẫn đến $h(n) > h^*(n)$ (vi phạm tính admissible).

**Câu 3.** Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

Đúng / Sai: Đúng

Giải thích: Euclidean distance là khoảng cách đường thẳng ("chim bay") giữa 2 điểm trên mặt phẳng. Vì đường đi ngắn nhất thực tế của Pacman trong mê cung luôn lớn hơn hoặc bằng đường thẳng, nên $h(n) \leq h^*(n)$ luôn đúng.

**Câu 4.** Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: Sai

Giải thích: Phản ví dụ: Giả sử $h_1(n)$ và $h_2(n)$ đều là các ước lượng admissible và $h_1(n) = h^*(n)$, $h_2(n) = h^*(n)$. Khi đó tổng $h_1(n) + h_2(n) = 2h^*(n) > h^*(n)$, vi phạm điều kiện $h(n) \leq h^*(n)$.

**Câu 5.** Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: Sai

Giải thích: Chiều ngược lại mới đúng (Consistency => Admissibility). Có những heuristic chấp nhận được (admissible) nhưng lại vi phạm bất đẳng thức tam giác giữa các đỉnh kề nhau (như bài tập 3.3 ở trên).

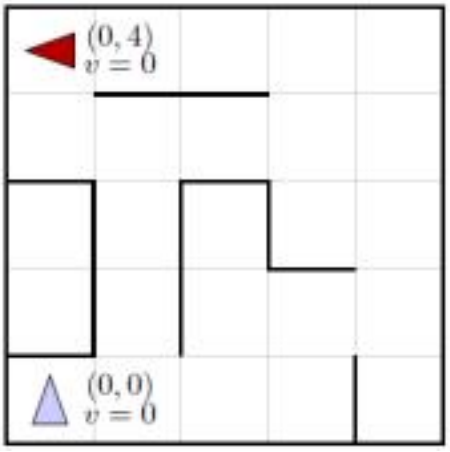
**Câu 6.** A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: Sai

Giải thích: Để $A^*$ dùng Graph Search (có giữ tập explored để không duyệt lại nút) đảm bảo tính tối ưu, heuristic phải đạt điều kiện Consistent (nhất quán). Nếu heuristic chỉ Admissible mà không Consistent, Graph Search có thể bỏ qua đường đi tối ưu đến một đỉnh đã nằm trong Explored Set.

# BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

## Câu 1. Tìm kiếm và heuristic




Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ {Bắc, Nam, Đông, Tây}. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

• Tăng tốc (Faster): v' = v + 1

• Giảm tốc (Slower): v' = v - 1

• Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận

tốc vẫn bằng 0.

• Rẽ trái: quay 90° ngược chiều kim đồng hồ.

• Rẽ phải: quay 90° theo chiều kim đồng hồ.

**Ví dụ**: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

• Hành động làm tác tử va vào tường.

• Làm v < 0 hoặc v > Vmax.



• Giữ tốc độ khi v = 0.

• Rẽ khi $v \neq 0$.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn (v = 0) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

**a)** Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

**Trạng thái ban đầu:** *(0,0)*, hướng bắc, $v = 0$.
**Trạng thái đích:** *(0,4)*, hướng tây, $v = 0$.
**Lộ trình:** *(0,0) → (0,1) → (0,2) → (1,2) → (0,2) → (0,3) → (0,4)*.
**Chuỗi hành động ngắn nhất (7 bước):**

3. Tăng tốc (Faster) => v'=1, di chuyển lên ô *(0,1)*, hướng bắc.
4. Giữ tốc độ (Maintain) => v'=1, di chuyển lên ô *(0,2)*, hướng bắc.
5. Rẽ phải (Turn Right) => hướng sang đông, $v = 1$ (Rẽ làm $v' = 0$).
6. Tăng tốc (Faster) => v'=1, di chuyển sang ô *(1,2)*, hướng đông.
7. Rẽ trái (Turn Left) => quay *180°* về hướng tây, $v = 0$.
8. Tăng tốc (Faster) => v'=1, di chuyển sang ô *(0,2)*, hướng tây.
9. Tăng tốc (Faster) => v'=2, di chuyển qua *(0,3)* đến ô *(0,4)*, dừng hẳn ($v = 0$) tại đích theo hướng tây

**b)** Nếu lưới có kích thước M × N và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

Một trạng thái *s* được xác định bởi 4 yếu tố:

- Vị trí $x \in \{0, 1, \ldots, M-1\}$($M$ vị trí).
- Vị trí $y \in \{0, 1, \ldots, N-1\}$ ($N$ vị trí).
- Hướng ∈ {bắc, nam, đông, tây} (*4* hướng).
- Vận tốc $v \in \{0, 1, \ldots, V_{max}\}$ ($V_{max} + 1$ giá trị).

Kích thước không gian trạng thái = $M \times N \times 4 \times (V_{max} + 1)$

**c)** Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

☑ Có ☐ Không

Xét trạng thái tác tử ở vị trí ngay trước bức tường (ví dụ *(0,1)* hướng bắc) với vận tốc $v = 1$.

- Do $v > 0$, tác tử không thể rẽ (chỉ rẽ khi $v = 0$).
- Với hành động giảm tốc ($v' = 0$), tác tử vẫn phải di chuyển $v' = 0$ ô, nhưng nếu việc di chuyển va vào tường trước đó thì không hợp lệ.
- Với hành động giữ tốc độ ($v' = 1$) hoặc tăng tốc ($v' = 2$), tác tử lập tức va vào tường ở ô *(0,2)* => tất cả các hành động đều không hợp lệ, do đó trạng thái này không có nút con nào.

**d)** Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

**Hệ số phân nhánh lớn nhất:** $b_{max} = 3$.
**Ví dụ trạng thái:** *(0,0, bắc, 1)* ở khu vực trống.
**Các hành động có thể thực hiện:**

- Slower ($v' = 0$)
- Maintain ($v' = 1$)
- Faster ($v' = 2$) (Khi $v = 0$, tác tử chỉ có 2 hành động hợp lệ là rẽ trái hoặc rẽ phải, do không được giữ tốc độ khi $v = 0$).

**e)** Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ☑ Không

Phản ví dụ:

- Giả sử tác từ đang ở cách cửa ra $d_M = 4$ ô theo khoảng cách Manhattan, hướng về phía cửa ra, vận tốc $v = 2$.
- Giá trị heuristic: $h(s) = 4$.
- Chi phí thực tế tối ưu đến đích: Chỉ cần 2 hành động:
    1. Maintain ($v' = 2$): di chuyển 2 ô (còn cách 2 ô).
    2. Slower ($v' = 1$): di chuyển 1 ô và dừng tại đích.
- Vì $h(s) = 4 > h^*(s) = 2$, heuristic này đánh giá cao hơn chi phí thực tế => Không admissible.

**f)** Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

☐ Có ☑ Không

Dù việc chia cho $V_{max}$ làm giảm đánh giá khoảng cách di chuyển, nhưng heuristic chỉ tính vị trí mà bỏ qua chi phí phải rẽ hướng và thời gian giảm tốc về 0 để dừng hẳn tại đích.

Phản ví dụ: Tác tử ở ô ngay cạnh đích ($d_M = 1$), $V_{max} = 5$, nhưng xe đang quay ngược hướng với đích và $v = 0$.

- Giá trị heuristic $h(s) = 1/5 = 0.2$.
- Chi phí thực tế tối ưu: Cần rẽ $180^o$ (2 hành động rẽ) + tăng tốc + giảm tốc = 4 hành động.
- Nếu $d_M = 0$ (đã đến đúng ô nhưng ngược hướng), $h(s) = 0$, nhưng chi phí thực tế $h^*(s) > 0$.

**g)** Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☑ Không

Giải thích: Trên đồ thị hữu hạn với chi phí không âm, $A^*$ vẫn sẽ duyệt qua tất cả các đường đi khả thi nếu cần cho đến khi tìm thấy đích (hoặc kiệt sức frontier) do $g(n)$ liên tục tăng, trừ khi $h(n) = \infty$ trên mọi đường đi. Vì vậy tính đầy đủ không bị thay đổi (vẫn Complete)

**h)** Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.



☑ Có ☐ Không

Chi phí đánh giá vượt quá chi phí thực tế ($h(n) > h^*(n)$) có thể khiến $A^*$ bỏ qua nhánh chứa lời giải tối ưu và trả về một đường đi dưới tối ưu (sub-optimal). Vì vậy tính tối ưu có bị thay đổi (không còn đảm bảo optimal)

**i)** Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

☑ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

☑ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☑ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

## Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có".

**a)** Breadth-First Search (BFS).

**Bất đẳng thức:**

$$\boldsymbol{d(n) < d(n_d)} \text{ hay } \boldsymbol{d(n) \leq d^* - 1}$$

**b)** Uniform-Cost Search (UCS).

**Bất đẳng thức:**

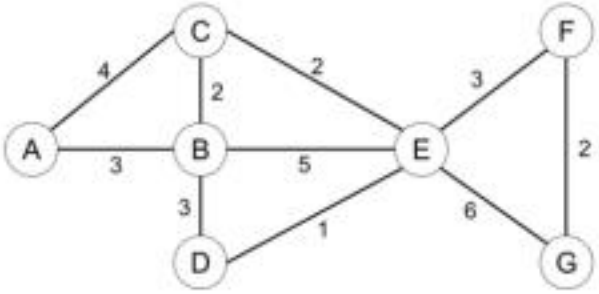
$$g(n) < C^*$$

**c)** A* Search, giả sử h là heuristic nhất quán.

**Bất đẳng thức:**

$$f(n) = g(n) + h(n) < C^*$$

## Câu 3. Tìm kiếm trên đồ thị




| Node | $h_1$ | $h_2$ |
|---|---|---|
| A | 12 | 11 |
| B | 6 | 7 |
| C | 9 | 6 |
| D | 3 | 4 |
| E | 3 | 5 |
| F | 2 | 1 |
| G | 0 | 0 |

| Nút | h1 | h2 |
|---|---|---|
| A | 12 | 11 |
| B | 6 | 7 |
| C | 9 | 6 |
| D | 3 | 4 |



| | | |
|---|---|---|
| E | 3 | 5 |
| F | 2 | 1 |
| G | 0 | 0 |

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá

hòa (tie-breaking).
Đồ thị vô hướng từ $A$ đến $G$:

- Các cạnh: $A-B:4, A-C:2, B-D:3, C-D:2, C-E:3, D-E:5, D-G:6, E-F:3, F-G:1$.

Chi phí thực tế các đường đi tới $G$:

- Path 1 ($A-C-E-G$): Cạnh $E-G$ không tồn tại trực tiếp (phải qua $F$).
- Path 2 ($A-C-E-F-G$): Cost = $2+3+3+1=9$.
- Path 3 ($A-B-D-E-F-G$): Cost = $4+3+5+3+1=16$.
- Path 4 ($A-C-D-G$): Cost = $2+2+6=10$.
- Path 5 ($A-B-D-G$): Cost = $4+3+6=13$.

**a)** Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

| Thuật toán | A-C-E-G | A-C-E-F-G | A-B-D-E-F-G | Khác |
|---|---|---|---|---|
| UCS | ☐ | ☑ | ☐ | ☐ |
| Greedy với h1 | ☐ | ☐ | ☐ | A-C-D-G |
| Greedy với h2 | ☐ | ☑ | ☐ | ☐ |
| A* với h1 | ☐ | ☑ | ☐ | ☐ |
| A* với h2 | ☐ | ☑ | ☐ | ☐ |

Greedy $h_1$: $A(12) \rightarrow C(9) \rightarrow D(3) \rightarrow G(0)$ => Đường đi $A-C-D-G$.
Greedy $h_2$: $A(11) \rightarrow C(6) \rightarrow E(5) \rightarrow F(1) \rightarrow G(0)$ => Đường đi $A-C-E-F-G$

**b)** Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

**Đáp án:** 9 (tương ứng với đường đi $A-C-E-F-G$).

**c)** Heuristic h1 có admissible không? Có consistent không?

Admissible: ☐ Có ☑ Không

Consistent: ☐ Có ☑ Không

**d)** Heuristic h2 có admissible không? Có consistent không?

Admissible: ☑ Có ☐ Không

Consistent: ☐ Có ☑ Không

## Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

**a)** Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ☑ Sai

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☑ Đúng ☐ Sai

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☑ Đúng ☐ Sai

Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

☐ Đúng ☑ Sai

Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

☐ Đúng ☑ Sai

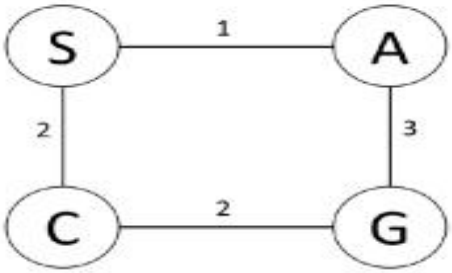
Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.

☑ Đúng ☐ Sai

**b)** Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.



| Trạng thái | h(s) |
|---|---|
| S | 4 |
| A | 3 |
| C | 3 |
| G | 0 |




| Heuristic Function | |
|---|---|
| State | $h(s)$ |
| $S$ | |
| $A$ | |
| $C$ | |
| $G$ | 0 |